MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 13

REPORT ON: TASK REPORT

TASK NO: 01-03

TASK 1 – ACCESSIBILITY AUDIT AND IMPROVEMENT PLAN

Introduction

Accessibility is an important aspect of modern application development. In this task, an accessibility audit is performed on an existing Compose UI screen to ensure that the interface can be used by all users, including those using assistive technologies such as screen readers. The audit evaluates screen reader compatibility, color contrast, touch target size, motion handling, and text scalability.

Selected UI Screen

The selected UI screen is the **Animated Quotes List Screen** created in previous sessions. The screen contains images, text descriptions, favourite buttons, and animated card layouts.

Accessibility Audit Checklist

Accessibility Item	Status	Evidence
Image content descriptions	Need Improvement	Some images missing contentDescription
Touch target size	Pass	Buttons larger than 48dp
Colour contrast	Need Improvement	Some text contrast is low
Motion reduction	Need Improvement	Animations always enabled
Focus order	Pass	Logical navigation through items
Text scaling	Pass	Layout adjusts for larger fonts

Improvement Plan

Issue	Fix	Priority
Missing image descriptions	Add contentDescription for images	High
Low color contrast	Use Material colorScheme.onPrimary	High
Touch targets	Apply Modifier.sizeIn(minHeight = 48.dp)	Medium
Motion settings	Respect system “Reduce Motion” setting	Medium

Example improvement code

```
Image(
    painter = painterResource(id = quote.imageResId),
    contentDescription = "Quote illustration"
)
```

Accessibility Testing Plan

Two automated accessibility tests will be implemented.

Example:

```
onNodeWithContentDescription("Quote illustration").assertIsDisplayed()
```

Another test:

```
onNodeWithText("Favourite").assertIsDisplayed()
```

These tests help ensure UI elements remain accessible during future updates.

Reflection

Accessibility ensures that applications can be used by people with different abilities. Planning accessibility from the beginning improves usability and inclusiveness. During the audit process, the most common challenge discovered was missing content descriptions for images and ensuring proper color contrast.

TASK 2 – LARGE-SCALE UI FEATURE: SUPERHEROES DIRECTORY

Feature Description

The proposed feature is a **Superheroes Directory Application** that displays a large list of superheroes with images, descriptions, and special abilities. Users can search heroes by name, mark favourites, and view detailed information for each hero.

Data Model

```
data class Superhero(
    val id: Int,
    val name: String,
    val alias: String?,
    val powerSummary: String,
    val imageUrl: String,
    val isFavourite: Boolean
)
```

Sample Dataset

ID	Name	Alias	Power	Favourite
1	Superman	Clark Kent	Super strength	false
2	Batman	Bruce Wayne	Intelligence	true
3	Spider-Man	Peter Parker	Spider abilities	false
4	Iron Man	Tony Stark	Advanced armor	false
5	Captain America	Steve Rogers	Super soldier	false
6	Thor	Odinson	Thunder powers	true
7	Hulk	Bruce Banner	Super strength	false
8	Wonder Woman	Diana Prince	Amazon warrior	false
9	Flash	Barry Allen	Super speed	false
10	Doctor Strange	Stephen Strange	Mystic arts	false

UI Layout Plan

Heroes List Screen

Components

- Search bar
- Filter toggle (favourites)
- LazyColumn list

Wireframe

Search Bar
Filter: Favourite

[Hero Image] Hero Name
Alias
Short Description
♥ Favourite

Hero Detail Screen

Hero Image
Hero Name
Alias
Power Summary

[Favourite Button]
[Back Button]

State and Logic Flow

State variables

```
searchQuery: String
filterFavourite: Boolean
heroesList: List<Superhero>
selectedHero: Superhero?
displayedList: List<Superhero>
```

Filtering logic example

```
val displayedList = heroesList.filter { hero ->
    hero.name.contains(searchQuery, ignoreCase = true)
    && (!filterFavourite || hero.isFavourite)
}
```

Accessibility and Testing Strategy

Accessibility improvements

- Hero image contentDescription = "${hero.name} photo"
- Search field labelled for screen readers
- Favourite buttons clearly labelled

Test cases

1. Verify search filters list correctly
2. Verify favourite toggle updates UI
3. Verify hero detail screen loads correct data
4. Verify images contain contentDescription
5. Verify state persists after screen rotation

Performance Considerations

To support large datasets (50+ heroes):

- Use LazyColumn for efficient list rendering
- Use image loading library **Coil**
- Compress hero images
- Implement pagination if dataset grows

Reflection

Designing this feature is more complex than earlier list applications because it includes search functionality, filtering, favourites, and multiple screens. Accessibility and testing add additional considerations but greatly improve application reliability and usability.

TASK 3 – 30-DAY COMPOSE PROJECT ROADMAP

Project Vision

The project titled **Compose Heroes Hub** will be a mobile application where users can explore superheroes, search characters, mark favourites, and view detailed hero information with animated UI and accessibility support.

Milestone Plan

Days	Goal	Tasks	Outcome
Day 1–6	Project Setup	Create project, theme system, data model	Base structure ready
Day 7–12	List UI	Build heroes list UI with dataset	Scrollable list working
Day 13–18	Interaction	Implement search, filter, favourites	Interactive UI
Day 19–24	Detail Screen	Add hero detail screen and animations	Multi-screen navigation
Day 25–30	Testing & Accessibility	Add automated tests and performance optimization	Final polished application

Tools and Resources

Development tools

- Jetpack Compose
- Coil image loader

- Android Studio
- GitHub

Learning resources

- Android Developer documentation
- Compose codelabs
- Kotlin documentation
- UI testing guides

Risk and Mitigation Plan

Risk	**Solution**
Large dataset affects performance	Implement lazy loading
Animations slow on old devices	Reduce animation complexity
Accessibility issues	Perform regular accessibility audits

Reflection

This 30-day project provides a structured path to apply all previously learned Compose concepts including theming, lists, animations, accessibility, and testing. It will help strengthen practical development skills and build a complete functional Android application.